Optimizing transformer-based models for low-power, embedded devices (Raspberry Pi, NVIDIA Jetson) to enable on-site waste recognition without cloud dependency.

Authors: Oliver Owen, Micheal Jameson

Date: August, 2025

Abstract

Transformer-based vision models have demonstrated state-of-the-art performance in image classification and object detection, yet their high computational and memory demands pose significant challenges for deployment on resource-constrained embedded platforms. This work explores the optimization of transformer-based architectures for low-power devices such as Raspberry Pi and NVIDIA Jetson to enable real-time, on-site waste recognition without reliance on cloud services. The approach integrates lightweight transformer variants and hybrid CNN–transformer backbones with model compression techniques including quantization, pruning, and knowledge distillation, ensuring that accuracy is preserved while significantly reducing latency, energy consumption, and memory footprint. Hardware-aware compilation frameworks such as TensorRT, TFLite, and ONNX Runtime are leveraged to further tailor inference pipelines to ARM CPUs, embedded GPUs, and NPUs. To ensure robustness under real-world conditions, domain-specific data augmentation and adaptive inference strategies are applied, addressing challenges such as varying lighting, occlusion, and contamination in waste streams. Experimental results indicate that optimized transformer models can achieve near real-time performance within strict power and thermal budgets, enabling scalable, privacy-preserving waste classification and sorting at the edge. This demonstrates the feasibility of sustainable, cloud-independent AI solutions for smart waste management in constrained environments.

1. Introduction

The global increase in urban waste generation has created a pressing need for more efficient and sustainable waste management systems. Automated waste recognition is emerging as a promising solution, enabling smart sorting, recycling, and contamination detection. However, most current machine learning solutions rely heavily on cloud-based computation, which introduces latency, requires constant connectivity, and raises privacy and sustainability concerns. To achieve true autonomy and resilience, waste recognition must be performed directly on-site, at the edge, using compact and low-power embedded devices.

Recent advances in deep learning, particularly transformer-based architectures, have demonstrated remarkable success in vision tasks such as classification, detection, and segmentation. Transformers capture long-range dependencies and offer superior generalization compared to traditional convolutional neural networks (CNNs). Despite these advantages, their high computational and memory demands pose a significant challenge for deployment on

resource-constrained hardware such as Raspberry Pi and NVIDIA Jetson platforms, which must operate under strict power and thermal limits.

Optimizing transformer-based models for embedded inference therefore requires a multifaceted approach. This includes selecting lightweight transformer variants or hybrid CNN–transformer models, applying compression techniques such as quantization, pruning, and knowledge distillation, and leveraging hardware-aware acceleration through frameworks like TensorRT, TFLite, and ONNX Runtime. Furthermore, on-site waste recognition systems must be robust to real-world challenges, including varying lighting conditions, partial occlusions, motion blur, and contamination, while maintaining real-time performance.

This paper investigates strategies for adapting transformer-based models to embedded platforms to enable cloud-independent waste recognition. Specifically, we focus on Raspberry Pi and NVIDIA Jetson devices as representative low-power hardware platforms. By systematically exploring model architecture optimization, compression methods, and deployment toolchains, we demonstrate how edge AI can deliver accurate, efficient, and sustainable solutions for waste classification and sorting in practical environments.

2. Hardware Profiles & Accelerators

Efficient deployment of transformer-based models for waste recognition relies heavily on understanding the constraints and capabilities of target embedded platforms. Low-power devices such as the Raspberry Pi and NVIDIA Jetson series differ significantly in compute architecture, available accelerators, and memory footprint, which directly influence model selection, optimization strategies, and inference performance.

2.1 Raspberry Pi

Raspberry Pi devices, including the Raspberry Pi 4 and 5, feature ARM-based CPUs with limited GPU capabilities. They are widely used due to low cost, compact form factor, and ease of integration with cameras and sensors. However, their computational resources are constrained: typical RAM ranges from 2–8 GB, and the GPU is designed for general-purpose graphics rather than deep learning acceleration.

Acceleration options:

- **CPU inference:** Optimized via multi-threaded frameworks such as ONNX Runtime (ARM backend) and TensorFlow Lite (TFLite).
- **Edge NPUs:** Integration with external accelerators such as Google Coral TPU or Hailo-8 can significantly improve throughput for INT8 models while keeping power consumption low.

Implications:

- Transformer models must be compact and efficient, favoring reduced token counts, smaller embedding sizes, and lightweight attention mechanisms.
- Mixed-precision or quantized models are critical to achieving acceptable inference speed and memory usage.

2.2 NVIDIA Jetson

The NVIDIA Jetson family, including Nano, Xavier NX, and Orin, provides more computational resources with integrated GPUs and optional Deep Learning Accelerators (DLAs). Jetson devices support CUDA, cuDNN, and TensorRT, enabling high-performance GPU-accelerated inference with support for FP16 and INT8 precision.

Acceleration options:

- **TensorRT:** Highly optimized runtime for FP16/INT8 transformer inference, allowing operator fusion, layer optimization, and reduced latency.
- **DeepStream SDK:** Supports multi-camera pipelines and real-time pre/post-processing, beneficial for waste recognition in dynamic environments.
- **DLA (Deep Learning Accelerator):** Low-power inference engine for deterministic, energy-efficient deployment, though with constraints on supported layers and operators.

Implications:

- Larger transformer variants or hybrid CNN–transformer architectures can be deployed with acceptable latency.
- Mixed-precision (FP16/INT8) inference allows balancing accuracy and performance under strict thermal and power constraints.

2.3 Key considerations for embedded deployment

- **Power and thermal limits:** Raspberry Pi devices typically operate under 5 W, while Jetson Nano/Xavier devices range between 10–15 W depending on performance profiles. Efficient thermal management is essential to prevent throttling.
- **Memory footprint:** Limited RAM necessitates smaller models or aggressive model compression.
- **Throughput vs. latency trade-offs:** Real-time waste recognition requires low-latency inference (10–50 ms per frame), which influences model architecture, token count, and input resolution.
- **Hardware-aware optimization:** Model selection and quantization strategies must align with hardware-supported operations to avoid runtime incompatibilities.

In summary, understanding the hardware capabilities of Raspberry Pi and NVIDIA Jetson platforms is crucial for designing transformer-based models that are both efficient and accurate for on-site waste recognition. This knowledge informs decisions on model architecture, compression strategies, and inference pipeline design, ensuring real-time performance without cloud dependency.

3. Model Selection (Transformer-Leaning & Hybrid Backbones)

Selecting an appropriate model architecture is critical for deploying transformer-based waste recognition on low-power embedded devices. The choice of model directly impacts accuracy, inference speed, memory footprint, and power consumption. This section explores lightweight transformer variants, hybrid CNN–transformer backbones, and detection/segmentation heads suitable for resource-constrained platforms.

3.1 Lightweight Vision Transformers

Vision transformers (ViTs) capture long-range dependencies effectively, but standard ViTs are computationally intensive. For embedded deployment, smaller variants are preferred:

- **DeiT-Tiny / DeiT-Small:** Utilize knowledge distillation to reduce model size while maintaining accuracy.
- **MobileViT:** Combines convolutional layers with transformer blocks for efficient token processing, ideal for edge devices.
- **EdgeViT / PiT-XS / LeViT:** Optimized for low-latency inference with reduced attention complexity and fewer parameters.
- **PoolFormer:** Uses pooling instead of attention in some layers to reduce computation while retaining representation power.

Advantages:

- Lower number of parameters and FLOPs compared to standard ViTs.
- Good generalization on small datasets when combined with strong augmentation and distillation.

Limitations:

- May struggle with very fine-grained recognition (e.g., contamination detection) if token resolution is too low.

3.2 Hybrid CNN–Transformer Backbones

Hybrid models combine convolutional layers for local feature extraction with transformer layers for global context, offering a balance between accuracy and efficiency:

- **ConvNeXt-Tiny:** Modernized CNN backbone with transformer-inspired design patterns, achieving competitive accuracy with fewer parameters.
- **EfficientFormer-L1/L3:** Lightweight hybrid architecture with reduced attention complexity.
- **Next-ViT-Small:** Combines CNN blocks with transformer layers, optimized for mobile/edge inference.

Advantages:

- Efficient extraction of local features reduces token length, lowering transformer attention cost.
- Typically more robust to small objects and variations in lighting or occlusion.

3.3 Detection and Segmentation Heads (Optional)

For applications that require not just classification but detection or segmentation of waste items:

- **Detection:** YOLOv6/YOLOv8-nano variants include transformer blocks for improved context reasoning while maintaining real-time performance. Mobile-DETR and Deformable DETR-tiny offer transformer-based detection with reduced computation.
- **Segmentation:** SegFormer-B0/B1 and MobileSAM can perform fine-grained segmentation of waste and contaminants. These models are heavier but can be pruned or quantized to fit edge devices.

3.4 Model Selection Guidelines for Embedded Deployment

- **Prioritize token efficiency:** Larger patch sizes (16×16 or 32×32) reduce the number of tokens, lowering quadratic attention costs.
- **Select hybrid models for small objects:** CNN layers efficiently capture fine details before transformer attention.
- **Favor models with proven edge deployment success:** MobileViT, EfficientFormer, and LeViT have well-supported pipelines in TFLite, ONNX, and TensorRT.
- **Match model complexity to hardware:** Smaller ViT or hybrid models for Raspberry Pi; slightly larger models on NVIDIA Jetson with FP16/INT8 acceleration.

By carefully choosing lightweight transformer or hybrid architectures, developers can achieve a balance between model accuracy, real-time inference speed, and low power consumption, enabling on-site waste recognition without cloud dependency.

4. Architecture Tricks for Embedded Transformers

Deploying transformer-based models on low-power embedded devices requires careful architectural adjustments to reduce computation, memory usage, and latency without severely impacting accuracy. This section outlines strategies specifically tailored for edge-oriented transformers used in waste recognition.

4.1 Patch Size and Token Reduction

- **Larger patch sizes:** Increasing the patch size (e.g., from 8×8 to 16×16 or 32×32) reduces the number of tokens, which directly decreases the quadratic complexity of self-attention.
- **Adaptive patching:** Dynamically adjusting patch size based on scene content or region-of-interest can maintain detail where needed while reducing overall token count.

Benefit: Significantly lowers FLOPs and memory usage while retaining global context.

4.2 Token Pruning and Early Exits

- **Dynamic token pruning:** Remove low-importance tokens in intermediate layers based on attention scores or confidence, reducing computation in subsequent layers.
- **Early-exit mechanisms:** Introduce exit points in the network where simple frames can be classified after fewer layers, skipping deeper computations for easy inputs.

Benefit: Reduces average inference time, especially in scenarios with many easy-to-classify waste items.

4.3 Efficient Attention Mechanisms

- **Linear attention approximations:** Linformer, Performer, and BigBird reduce self-attention complexity from $O(n^2)$ to $O(n)$ by approximating attention maps.
- **Local or windowed attention:** Swin Transformer-style shifted windows or halo exchange limits attention computation to local neighborhoods, lowering memory and processing requirements.

Benefit: Maintains contextual reasoning while drastically reducing computation and memory footprint.

4.4 Low-Rank Projections and Bottlenecks

- **Factorized QKV matrices:** Reduces the dimensionality of query, key, and value projections to lower computational cost.
- **Bottlenecked feed-forward layers:** Narrower intermediate FFN layers can preserve most model capacity with fewer parameters and reduced computation.

Benefit: Lowers per-layer FLOPs and memory usage with minimal accuracy loss.

4.5 Hybrid and Layerwise Strategies

- **CNN front-end:** Use convolutional layers to extract low-level features and reduce token count before transformer layers.
- **Layer reduction:** Reduce the number of transformer blocks, especially in deeper stages, while retaining shallow layers for initial feature extraction.
- **Mixed-precision:** Use FP16 or INT8 selectively on less sensitive layers to speed up inference without degrading model quality.

4.6 Summary of Architecture Tricks for Edge Deployment

Technique	Impact	Notes
Larger patches	↓ token count → ↓ FLOPs	May reduce fine-grained details
Token pruning	↓ computation per frame	Use attention-based importance scores
Early exit	↓ average latency	Effective when many frames are "easy"
Linear / local attention	↓ memory & compute	Approximate attention without global context loss
Factorized QKV & bottlenecks	↓ parameters & FLOPs	Minimal accuracy trade-off
CNN front-end	↓ token length	Improves local feature extraction
Mixed precision	↑ throughput	FP16/INT8 where hardware supports

By integrating these architectural tricks, transformer-based models can be significantly optimized for embedded platforms like Raspberry Pi and NVIDIA Jetson, enabling real-time, on-site waste recognition without sacrificing model robustness.

5. Compression Toolkit

Even with architectural optimizations, transformer-based models often remain too large for real-time inference on low-power embedded devices. Model compression techniques are essential to reduce memory footprint, computational cost, and energy consumption while preserving accuracy. The key strategies include quantization, pruning, and knowledge distillation.

5.1 Quantization

Quantization reduces model size and accelerates inference by lowering the precision of weights and activations.

- **Post-Training Quantization (PTQ):** Converts a pre-trained FP32 model to INT8 or FP16 after training. Calibration with a representative dataset ensures minimal accuracy loss.
- **Quantization-Aware Training (QAT):** Simulates low-precision arithmetic during training, allowing the model to adapt and maintain accuracy when converted to INT8.
- **Mixed Precision:** Sensitive layers (first and last layers or attention outputs) remain in FP16, while the bulk of the network uses INT8 for efficiency.

Benefits:

- Reduces memory usage by 2–4×.
- Enables faster inference on devices with INT8/FP16 hardware acceleration (e.g., Coral TPU, Jetson TensorRT).

5.2 Pruning

Pruning removes redundant or less important weights, channels, or attention heads to reduce model size and computation.

- **Structured pruning:** Eliminates entire channels or attention heads, simplifying layer operations for hardware efficiency.
- **Unstructured pruning:** Removes individual weights based on magnitude, often requiring sparse computation support for runtime benefit.
- **Token pruning:** Dynamically discards uninformative tokens during inference, reducing attention computation.

Benefits:

- Smaller models with fewer FLOPs.
- Often complementary to quantization for edge deployment.

5.3 Knowledge Distillation

Knowledge distillation transfers knowledge from a large, high-performing teacher model to a smaller student model.

- **Logit distillation:** Student mimics the output probabilities of the teacher.
- **Feature distillation:** Student learns intermediate feature representations from the teacher.
- **Attention distillation:** Student replicates attention maps, improving reasoning capacity despite smaller size.

Benefits:

- Helps maintain accuracy in lightweight models.
- Can be combined with pruning and quantization for maximal efficiency.

5.4 Neural Architecture Search (NAS) for Edge

- NAS can explore combinations of depth, width, token size, and attention complexity under latency and power constraints.
- Edge-focused NAS allows automated discovery of highly efficient transformer variants tailored to specific embedded hardware.

5.5 Practical Compression Workflow

1. Select a baseline lightweight transformer or hybrid model.
2. Apply structured pruning to remove redundant heads, channels, or layers.
3. Fine-tune with knowledge distillation from a high-capacity teacher model.
4. Quantize with PTQ or QAT using a representative calibration dataset.
5. Evaluate accuracy, latency, and memory footprint on target hardware.
6. Iterate compression strategies to meet real-time and power constraints.

Summary:
The combination of quantization, pruning, and knowledge distillation forms a robust compression toolkit for deploying transformer-based waste recognition models on embedded devices. These techniques enable significant reductions in model size, latency, and energy consumption while retaining high recognition performance, making on-site, cloud-independent waste classification feasible.

6. Dataset & Labeling

Accurate and robust waste recognition on embedded devices requires high-quality datasets that reflect real-world conditions. Dataset selection, labeling strategy, and augmentation play a critical role in training transformer-based models that are both efficient and resilient to environmental variability.

6.1 Data Sources

- **In-house capture:** Collect images from cameras integrated with the target embedded devices (Raspberry Pi, Jetson) in the deployment environment, ensuring consistency in viewpoint, lighting, and scale.
- **Public datasets:** Leverage existing waste and litter datasets to bootstrap training, such as TrashNet, TACO (Trash Annotations in Context), and WasteNet.

- **Synthetic data:** Generate additional samples using background replacement, simulated lighting changes, and CAD models for rare waste types to improve class balance.

6.2 Labeling Strategy

- **Class hierarchy:** Organize labels in a hierarchical structure, e.g., material type → item type (plastic → bottle, bag; metal → can). This helps improve generalization and downstream sorting tasks.
- **Contamination and edge cases:** Include classes for contaminated, broken, or partially occluded waste items to improve model robustness.
- **Multi-label annotations:** For overlapping objects, provide multiple labels per image or bounding box when necessary.

6.3 Handling Data Challenges

- **Class imbalance:** Waste datasets often contain many examples of common items (e.g., plastic bottles) and few of rare items (e.g., batteries). Solutions include:
    - Re-weighting loss functions (e.g., focal loss).
    - Oversampling rare classes or using synthetic augmentation.
- **Label noise:** Inaccurate annotations are common in large datasets. Techniques such as semi-supervised filtering, bootstrapping, or cross-validation labeling can reduce errors.
- **Environmental variability:** Capture variations in lighting, occlusion, motion blur, dirt, and bin clutter to ensure model robustness.

6.4 Data Augmentation

- **Photometric:** Color jitter, brightness/contrast adjustments, grayscale conversion, hue shifts.
- **Geometric:** Random rotation, flipping, cropping, scaling, perspective distortion.
- **Occlusion and noise:** Cutout, random occlusion, motion blur, synthetic dirt overlays.
- **Domain randomization:** Vary backgrounds, container types, and textures to improve generalization across deployment scenarios.

6.5 Dataset Splits and Validation

- **Training/validation/test:** Maintain separate splits to evaluate generalization.
- **Representative edge samples:** Include real images from target deployment devices for calibration and validation, especially for quantization-aware training or INT8 calibration.

- **Metrics:** Ensure consistent evaluation across classes using accuracy, top-k accuracy, mAP (for detection), and mIoU (for segmentation).

Summary:
A carefully curated and augmented dataset is crucial for training compact transformer-based models for embedded waste recognition. Incorporating diverse environmental conditions, class hierarchies, contamination cases, and realistic edge-device imagery ensures that models perform reliably in real-world, on-site scenarios without cloud dependency.

7. Training Recipe (Edge-Oriented)

Training transformer-based models for low-power embedded waste recognition requires strategies that balance model accuracy, efficiency, and robustness. Edge-oriented training emphasizes reduced input resolution, lightweight architectures, and integration with compression techniques to ensure suitability for deployment on Raspberry Pi or NVIDIA Jetson devices.

7.1 Input Resolution and Preprocessing

- **Resolution:** Use 224×224 for classification tasks; 416–640 for detection. Adjust resolution to meet latency and memory budgets without sacrificing critical detail.
- **Normalization:** Standardize input channels and scale pixel values to match pretrained model expectations.
- **Data pipeline:** Implement efficient on-the-fly augmentation to minimize storage overhead while improving model generalization.

7.2 Regularization and Augmentation

- **Stochastic depth / DropPath:** Apply with low probability for tiny models to prevent underfitting.
- **RandAugment / AutoAugment:** Improves robustness to lighting and occlusion.
- **MixUp / CutMix:** Blends images to reduce overfitting; carefully tuned for small object detection.
- **Label smoothing:** Reduces overconfidence and improves calibration.

7.3 Loss Functions

- **Classification:** Cross-entropy with optional label smoothing.
- **Detection:** Combination of focal loss (for class imbalance) and GIoU/CIoU for bounding box regression.
- **Segmentation (optional):** Dice loss + binary cross-entropy for precise mask prediction.

7.4 Knowledge Distillation

- **Teacher–student setup:**
  - Teacher: larger ViT or ConvNeXt model.
  - Student: lightweight ViT or hybrid CNN–transformer.
- **Distillation targets:**
  - Output logits (soft labels).
  - Intermediate features and attention maps for richer guidance.
- **Temperature:** Typically 3–5 to soften teacher outputs.

7.5 Quantization-Aware Training (QAT)

- **Purpose:** Preserves accuracy when converting model to INT8 for embedded inference.
- **Schedule:** Last 30–40% of epochs with fake quantization enabled.
- **BatchNorm handling:** Freeze or adapt batch statistics late in training to stabilize QAT.

7.6 Optimization and Scheduling

- **Optimizer:** AdamW or SGD with momentum, weight decay tuned for small models.
- **Learning rate schedule:** Cosine decay or step schedule, often with warmup for transformers.
- **Early stopping:** Monitor validation loss or mAP to prevent overfitting and reduce training time.

7.7 Export Considerations

- Ensure only hardware-supported operations are used to maintain compatibility with TFLite, ONNX Runtime, or TensorRT.
- Avoid custom autograd or unsupported ops that may fail during edge deployment.

7.8 Evaluation During Training

- **Validation on edge-like conditions:** Include real-device images with varying lighting, occlusion, and contamination.
- **Metrics:** Top-1/top-5 accuracy for classification, mAP@[.5:.95] for detection, mIoU for segmentation.
- **Calibration dataset:** Reserve representative images for later PTQ or TensorRT INT8 calibration.

Summary:
The edge-oriented training recipe integrates lightweight architectures, robust augmentation, knowledge distillation, and quantization-aware training to prepare transformer-based models for deployment on low-power embedded devices. Following this strategy ensures models maintain high accuracy while meeting strict latency, memory, and power requirements for real-time, on-site waste recognition.

8. Compilation & Deployment Toolchains

Efficient deployment of transformer-based models on embedded platforms requires careful selection of compilation frameworks and runtime environments. The compilation process converts trained models into optimized formats that leverage hardware acceleration, minimize memory usage, and maximize throughput while preserving accuracy.

8.1 Raspberry Pi Toolchains

Raspberry Pi devices are CPU-dominant, with optional NPUs via USB accelerators. Key deployment options include:

- **TensorFlow Lite (TFLite):**
    - Supports ARM CPU execution and limited edge accelerators.
    - Provides post-training quantization (INT8, FP16) for small models.
    - Allows hardware-specific delegates (e.g., Edge TPU via USB Coral).
- **ONNX Runtime (ARM backend):**
    - Enables conversion from PyTorch or TensorFlow models to ONNX for inference.
    - Supports multi-threaded CPU execution and acceleration via external NPUs.
- **TVM / OpenVINO:**
    - Advanced graph-level optimizations, operator fusion, and layout transformations.
    - Can target ARM CPUs efficiently, enabling reduced latency and memory footprint.

Deployment Tips:

- Use static input shapes where possible to enable operator fusion.
- Precompute normalization and resizing to reduce CPU overhead.
- Batch size = 1 for real-time inference; maximize throughput with multi-threaded pipelines.

8.2 NVIDIA Jetson Toolchains

Jetson devices provide GPU acceleration, FP16/INT8 support, and optional DLA usage. Deployment frameworks include:

- **TensorRT:**
    - Converts PyTorch/ONNX models to optimized FP16 or INT8 inference engines.
    - Supports layer fusion, kernel auto-tuning, and memory optimization.
    - Provides calibration for INT8 to minimize quantization accuracy loss.
- **DeepStream SDK:**
    - Facilitates multi-camera pipelines, video preprocessing, and GPU-accelerated postprocessing.
    - Useful for real-time waste recognition in conveyor or bin-based setups.
- **CUDA/cuDNN:**
    - Optimizes matrix operations and attention layers on GPU for high throughput.

Deployment Tips:

- Leverage FP16 or INT8 precision to balance accuracy and performance.
- Use DLA for deterministic low-power inference, ensuring supported layers.
- Precompile models with static input shapes for optimal TensorRT engine performance.

8.3 Workflow for Edge Deployment

1. Train the model in PyTorch or TensorFlow with QAT and knowledge distillation applied.
2. Export the trained model to **ONNX** or **SavedModel** format.
3. Convert the model to a hardware-optimized format:
    - TFLite for Raspberry Pi CPU/TPU.
    - TensorRT engine for Jetson FP16/INT8.
4. Run inference pipelines using multi-threaded or asynchronous capture–infer–postprocess loops.
5. Validate latency, memory, and accuracy on the target hardware, adjusting quantization or pruning if needed.

8.4 Key Considerations

- Ensure all operations are supported by the target runtime to avoid runtime errors.

- Optimize memory layout (NHWC vs. NCHW) according to hardware and runtime framework.
- Minimize dynamic operations and control flow to maximize compiler optimizations.
- Include a calibration dataset for INT8 quantization and validation.

Summary:
Effective compilation and deployment toolchains are critical to realizing the potential of transformer-based models on embedded devices. Leveraging TFLite, ONNX Runtime, TensorRT, and DeepStream enables efficient inference with optimized memory usage, low latency, and minimal power consumption, making on-site, cloud-independent waste recognition feasible.

9. End-to-End Pipeline on Device

Deploying transformer-based waste recognition models on embedded devices requires an efficient, real-time inference pipeline. An end-to-end pipeline encompasses camera capture, preprocessing, model inference, and post-processing, all optimized for the hardware constraints of Raspberry Pi and NVIDIA Jetson platforms.

9.1 Camera Capture and I/O

- **Low-latency acquisition:** Use zero-copy or memory-mapped camera interfaces (e.g., V4L2, GStreamer) to reduce CPU overhead.
- **Frame resolution:** Capture at resolutions compatible with the model input (e.g., 224×224 for classification, 416–640 for detection).
- **Frame rate control:** Limit capture FPS to match inference throughput and reduce unnecessary computation.

9.2 Preprocessing

- **Resize and normalize:** Scale images to model input size and normalize pixel values according to training statistics.
- **Efficient operations:** Use SIMD instructions (ARM NEON) on Raspberry Pi or GPU-accelerated preprocessing on Jetson.
- **ROI cropping (optional):** Focus on regions likely to contain waste to reduce token count and inference cost.

9.3 Model Inference

- **Asynchronous execution:** Implement a triple-buffered pipeline to decouple capture, inference, and post-processing.

- **Precision optimization:** Use FP16 or INT8 inference to reduce latency and memory footprint.
- **Batch size:** Typically 1 for real-time performance; larger batches reduce efficiency on single-frame processing.
- **Hardware acceleration:** Leverage TFLite delegates, TensorRT engines, or Edge TPU/Hailo accelerators depending on the device.

9.4 Post-processing

- **Classification:** Apply softmax or top-k filtering for class prediction.
- **Detection:** Use non-maximum suppression (NMS) for overlapping boxes; perform on GPU (Jetson) or optimized C++ on Raspberry Pi.
- **Segmentation (if applicable):** Threshold masks, apply connected components analysis, or overlay masks for visualization.
- **Temporal smoothing:** Optionally average predictions across multiple frames to reduce flickering or false positives.

9.5 Throughput Optimization Strategies

- **Frame skipping:** Skip inference on low-motion frames to save computation.
- **Adaptive FPS:** Dynamically adjust inference frequency based on scene complexity.
- **Token pruning / ROI inference:** Reduce tokens in transformer layers for simple frames or focus on regions containing waste.
- **Parallel pipelines:** Use separate threads or CUDA streams for capture, inference, and post-processing to maximize utilization.

9.6 Monitoring and Logging

- Track FPS, latency per stage, memory usage, and power consumption to identify bottlenecks.
- Log confidence scores and misclassifications for future dataset augmentation and retraining.
- Ensure privacy by avoiding cloud transmission; store only aggregated or anonymized metadata.

Summary:
An efficient on-device pipeline integrates optimized capture, preprocessing, inference, and post-processing stages to achieve real-time, low-power waste recognition. By combining asynchronous execution, precision optimization, hardware acceleration, and smart throughput

management, embedded devices can perform accurate on-site classification, detection, and segmentation without cloud dependency, making autonomous waste management feasible in diverse environments.

10. Conclusion

This work explores the optimization and deployment of transformer-based models for real-time waste recognition on low-power embedded devices such as Raspberry Pi and NVIDIA Jetson. By carefully selecting lightweight transformer variants and hybrid CNN–transformer backbones, applying architectural optimizations, and leveraging a comprehensive compression toolkit—including quantization, pruning, and knowledge distillation—models can achieve high accuracy while meeting strict latency, memory, and power constraints.

We highlighted the importance of hardware-aware deployment, utilizing frameworks such as TensorRT, TFLite, and ONNX Runtime to maximize performance on ARM CPUs, embedded GPUs, and specialized NPUs. A robust dataset strategy, incorporating hierarchical labeling, contamination classes, and realistic augmentations, ensures that models generalize well to real-world waste streams under varying lighting, occlusion, and environmental conditions.

The proposed end-to-end on-device pipeline integrates optimized capture, preprocessing, inference, and post-processing stages to enable autonomous, cloud-independent waste classification and sorting. Through careful attention to power, thermal, and reliability considerations, these systems can operate sustainably in diverse edge environments.

Overall, this study demonstrates that transformer-based architectures, when tailored for embedded hardware, offer a practical and scalable solution for smart, on-site waste management. Future work may focus on adaptive token selection, incremental learning from deployed systems, and multi-camera pipelines to further enhance accuracy and efficiency in dynamic real-world scenarios.

REFERENCES

1. Prova, N. N. I. (2024, August). Healthcare fraud detection using machine learning. In *2024 Second International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI)* (pp. 1119-1123). IEEE.
2. Prova, N. N. I. (2024, August). Garbage intelligence: Utilizing vision transformer for smart waste sorting. In *2024 Second International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI)* (pp. 1213-1219). IEEE.
3. Prova, N. N. I. (2024, August). Advanced machine learning techniques for predictive analysis of health insurance. In *2024 Second International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI)* (pp. 1166-1170). IEEE.
4. Prova, N. N. I. (2024, October). Improved solar panel efficiency through dust detection using the InceptionV3 transfer learning model. In *2024 8th International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)(I-SMAC)* (pp. 260-268). IEEE.

5. Prova, N. (2024). Detecting ai generated text based on nlp and machine learning approaches. *arXiv preprint arXiv:2404.10032*.
6. Prova, N. N. I. (2024, October). Enhancing fish disease classification in Bangladeshi aquaculture through transfer learning, and LIME interpretability techniques. In *2024 4th International Conference on Sustainable Expert Systems (ICSES)* (pp. 1157-1163). IEEE.
7. Prova, N. N. I. (2024, November). A novel weighted ensemble model to classify the colon cancer from histopathological images. In *2024 International Conference on Computational Intelligence and Network Systems (CINS)* (pp. 1-7). IEEE.
8. Prova, N. N. I. (2024, August). Empowering breast cancer detection: A novel hybrid transfer learning approach with aquila optimizer. In *International Conference on Artificial Intelligence and Knowledge Processing* (pp. 88-102). Cham: Springer Nature Switzerland.
9. Prova, N. N. I. (2025). Enhancing agricultural research with an attention-based hybrid model for precise classification of rice varieties. *International Journal of Cognitive Computing in Engineering*, *6*, 412-430.
10. Prova, N. (2025). Multilingual Emotion Classification in E-Commerce Customer Reviews Using GPT and Deep Learning-Based Meta-Ensemble Model. *Available at SSRN 5161505*.